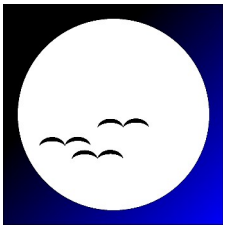

The QMdbSim2 Project

An Improved Microchip Device Simulator Framework For QSpice

Document History

2026.06.29 – Initial documentation. Caveat emptor.

Notes

All projects mentioned in this document along with source code and examples are available on my QSpice GitHub repository.

I assume that you have experience using the Microchip MPLabX tools to create micro-controller device code (*.elf/*.hex files).

Parts of this project use Java. Pre-compiled JAR files are provided so you don't need to know anything about Java unless you wish to modify the simulator interface code. However, you will need to install a 32-bit Java 8+ JRE to run simulations.

Project Overview

The QMdbSim2 Project is a framework for using Microchip micro-controller devices in QSpice simulations. You embed a micro-controller symbol in your project, provide the name of your compiled device code (`*.elf`/`*.hex`), and, when you run the QSpice simulation, your device code drives the device symbol pins in the schematic circuit.

Under the covers, the project uses Microchip's software simulator SDK (MDBCore) so it supports the same PIC and AVR devices to the same degree that Microchip's software debugger supports the device. (See "Limitations" below.) Examples for `PIC16F1521x` and `ATtiny85` devices are provided but you can easily add other devices using the `QSymGen3` tool from the `QParser2` project.

The project is easy to use but, underneath the covers, it's rather complex. This document includes:

- A high-level architecture overview.
- Terminology used in this project.
- Definitions of "user types" (i.e., Basic User, Device Developer, and API Developer), what they do, tools required, and pointers to other documents relevant to those activities.
- `QMdbSim2` limitations.
- Frequently Asked Questions.

Note that I assume that the reader is familiar with Microchip micro-controller devices generally. You should be familiar with Microchip's MPLabX IDE.

Architecture & Terminology

At a high level, a `QMdbSim2`-based project looks like this (I suggest that you open the mentioned `*.qsym` and `*.qsch` example files):

- A top-level QSpice schematic containing the circuitry connected to a `QMdbSim2` symbol. This symbol is "device-specific," created for a particular Microchip device of family of devices. For example, the project sample `PIC16F1521x.qsym` supports `PIC16F15213` & `PIC16F15214` devices.
- This *Device Symbol* contains a "clock pin" used to advance the device simulation in single-instruction steps under QSpice control.
- The Symbol Properties pane contains a drop-down list for selecting the specific device and edit boxes to specify paths to a configuration file (`QMdbCfg.ini`) and to the user's device-specific program. This "device program" is the user-supplied/compiled `*.elf`/`*.hex` program to load into the simulation.
- The Device Symbol is a hierarchical schematic block which loads a device-specific schematic (*Device Schematic*). For example, `PIC16F1521x.qsym` loads `PIC16F21x.qsch`. The Device Schematic is not intended for user modification. (In fact, there is a tool to generate this schematic, `QSymGen3`.)
- The Device Schematic contains a hierarchical DLL block which loads the *Device DLL*, e.g., `PIC16F1521x.dll`. This DLL does all of the magic to pass pin/port states between QSpice and the Java-based Microchip software simulator.
- This Device DLL loads a configuration file, `QMdbCfg.ini`, to obtain the paths to:
    - The MPLabX IDE installation location.
    - A 32-bit Java runtime environment (JRE or JDK).
    - An instance of the project's `QMdbCS.jar` file.
- The Device DLL launches Java and loads the `QMdbCS.jar` file. (Note: The JAR is device-agnostic and used by all `QMdbSim2` devices.)
- For the remainder of the QSpice simulation, the DLL passes pin/port states between QSpice and the Microchip software simulator on each trigger of the Device Symbol clock pin.

OK, I said "high level." And that was pretty high-level. I also said that it's easy to use. And it is, at least after the initial configuration. And there are various device-specific elements; where do they come from? Read on.

User Types

I'll classify users into three groups: ***Basic Users***, ***Device Developers***, and ***API Developers***.

Basic Users

Basic users use *existing* device-specific elements (symbols, schematics, DLLs) and common `QMdbSim2` project files to create top-level schematics. They write and compile Device Programs (`*.elf`/`*.hex`) to use in QSpice simulations. They must install certain tools (see below) and edit a configuration file to point to those tools. (This is one-time installation and doesn't change for different Microchip devices.)

Need a specific device but nobody has created one? It's easy to become a Device Developer.

Device Developers

Device Developers *create new* device-specific elements (symbols, schematics, DLLs) for Basic Users. The `QSymGen3` command-line tool (part of the `QParser2` project) makes it easy to create these bits from a Microchip datasheet. (Seriously, define the device characteristics and pins in a text file. Run `QSymGen3.exe` to generate the device-specific symbols, schematics, and DLL code. Compile the DLL code. Distribute.)

API Developers

API Developers modify the device-agnositic DLL-to-Java interface code. The DLL and `QMdbCS.jar` communicate using JNI for efficiency. One might want to combine multiple JNI calls into a custom message to reduce the non-negligible JNI call overhead. Other reasons include supporting very device-specific features that `QMdbSim2`'s simple set/step/fetch pin interface doesn't support.

Note: If you don't trust the pre-compiled DLL and/or JARs – and you shouldn't – then you'll want to become a Device/API developer.

Tool Requirements

Running a `QMdbSim2` project requires some initial/one-time setup and configuration. Here's a summary of requirements.

Tool	**Basic User**	**Device Developer**	**API Developer**
MPLabX v6.30 (for simulator API and Device Program development)	✓	✓	✓
32-bit Java v8 JRE or newer	✓	✓	✓
MSVS 2026 (for C++20 DLL compiles)		✓	✓
QSymGen3 command-line tool to create new device-specific files		✓	
Microchip device datasheet(s)		✓	
QMdbSim2 project files (for DLL compiles)		✓	
32-bit Java JDK, v8 or newer (to compile Device DLLs)		✓	✓

Tool	Basic User	Device Developer	API Developer
32- or 64-bit Java JDK, v8 or newer (to compile QMdbCS.jar)			✓
Microchip SDK			✓
Ant (recommended for QMdbCS.jar builds)			✓

Project Documentation

Additional project documentation is available in the GitHub project repository.

QMdbSim2_Basic_User.pdf	Initial installation and configuration. Basic usage.
QMdbSim2_Device_Dev.pdf	Device files overview. Creating new device-specific files using QSymGen3 project tools.
QMdbSim2_API_Dev.pdf	Back-end software simulator API. Overview and resources.

QMdbSim2 Limitations

`QMdbSim2` has some limitations:

- It uses Microchip's software simulator behind the scenes. The software simulator does not support all Microchip devices and, for a given device, may not support all functionality. For example, a given device may have an "ADC peripheral." The software simulator may not (probably doesn't) support the peripheral.

  If the Microchip simulator doesn't support the functionality then `QMdbSim2` doesn't support it. See the FAQs section for related information.

  You can find a table of supported functionality here: Simulator Peripheral Support By Device

- Similarly, some generic functionality is not supported. For example, a pin may have a software-configured weak pull-up resistor. It may contain an internal oscillator or pins to attach an external crystal oscillator. Again, maybe not supported.

- `QMdbSim2` devices are not "electrically correct." See FAQs for suggestions for more accurate electrical modeling.

Frequently Asked Questions (FAQs)

Where can I get support for the QMdbSim2 project?

The best place to contact me is on the Qorvo QSpice forum (@RDunn).

Can I compile sources with the DMC compiler shipped with QSpice?

Short Answer: No.

Long Answer: The Digital Mars C/C++ compiler (DMC) is ancient and support ended with C++98. The absence of a modern IDE and support for current C++ standards makes it unsuitable for projects of this complexity. That said, it may be possible to back-port the project code.

This project seems pretty complicated. Isn't there an easier way?

The original `QMdbSim` project was simpler – it used the Microchip `MDB.bat` command-line debugger included in the MPLabX installation. No additional Java tools required. That project is available on the GitHub repository.

That said, `QMdbSim2` simulations run much, much faster via Java Native Interface (JNI) calls. Developing new device-specific symbols, schematics, and DLLs is automated with `QSymGen3`. Configuration is now contained in the `QMdbCfg.ini` file.

Bottom line: After the initial setup, `QMdbSim2` is better in almost every way.

Why are simulations so slow?

Hmm. Well, QSpice simulations can run slowly for many reasons. I'll assume that you've ruled out the normal stuff (convergence issues and whatnot). That leaves us with the `QMdbSim2`-specific speed question.

The project uses Microchip software simulator for the back-end. This is Java code (aka "slow") and it emulates hardware (aka "slow"). That said, my tests suggest that the first 10-20+ seconds of a simulation are burned spinning up Java and initializing the simulator. A second simulation run generally starts up and executes much faster (presumably due to operating system caching).

That said, the DLL-to-JNI interface could be improved. I have some ideas for future releases. In the meantime, feel free to become an API Developer and send me improved code.

Is there an official repository for device-specific symbols and DLLs?

I'm setting aside a GitHub repository folder for community contributions. I hope that others will share.

Can I put two instances of a device symbol in the same schematic?
Can I run a simulation with multiple steps?

Excellent questions. I've not yet tested either case. They are on my post-first-release list to investigate. In the meantime, there's no harm in testing it yourself. (If you do, please let me know what you find.)

The performance timer times don't match QSpice simulation time. Why?

The embedded timers have, I think, micro-second resolution. They are not highly accurate and subject to rounding. They are intended solely as rough estimates of how much time is spent initializing the Java simulation vs time spent actually executing simulation Java code.

MPLabX includes a Java JRE. Why can't I use that? What Java versions can I use?

The Device DLL is a 32-bit process. It cannot launch a 64-bit Java instance. The MPLabX JRE is a 64-bit JRE so you must install a 32-bit Java JRE. Annoying but unavoidable.

You can use any 32-bit version of the Java JRE (or JDK) that supports Java 8 or later. I've used the Eclipse Adoptium Temurin 8 & 17 32-bit releases (see references). Java seems generally to be dropping support for 32-bit JRE so v17 might be the end of the line for new versions.

What peripheral devices are supported for XXX device?

Micro-controllers often support "peripheral devices" such as ADC converters, SPI/I^2C, USB, UART, WDT, interrupt timers, internal oscillators, etc. These may or may not be supported by the Microchip software simulator for your device. See below for a link to Microchip's table of simulator support. **[add linked references here and elsewhere]**

Bottom line: If Microchip's simulator doesn't support the functionality, this project doesn't either.

My XXX device has pull-up resistors that can be enabled in device software. When my Device Program enables/disables them, nothing changes. Why?

Whether or not a device pin supports internal pull-ups and which pins support them is highly device-specific. The project would require substantial additional circuitry and code to support device-specific characteristics such as this.

As far as I know, pins with "internal weak pull-up resistors" are usually open-collector/open-drain pins. You can add external transistors/resistors in the top-level schematic to electrically simulate a weak pull-up configuration. However, that's static – if you truly need to dynamically switch the pull-ups on/off in the Device Program, well, that's not something that we can do in the generic simulator interface. Maybe I'll add it in the future if someone gives me a compelling use-case.

Why does the electrical behavior of my XXX device not match the datasheet? For example, the current at the supply pin appears to be 0A and the current on output pins seems unlimited.

QSpice DLL blocks are very basic, ideal models. Input pins do not consume current. Output pins are voltage sources with a default internal resistance of 1K.

The project sets the internal resistance of output pins to 1Ω in the project Device Schematic using the ROUT instance parameter. Bi-directional pins add an additional 1Ω resistor in the default GPIO implementation (`GPIO_IMPL.qsch`). You can, of course, change these.

Why am I getting timestep too small errors?

The only case that I've encountered so far where this was actually related to using the `QMdbSim2` project was where I tied a Device Symbol output pin to a top-level schematic voltage source. (Bi-directional pins may start out as input pins and become output pins under Device Program control.)

If you find other sources of this error, please let me know and I'll document them here.

Why are messages in the QSpice Output window corrupted?

This is a long-term problem with QSpice. I'll not burden you with the underlying details but I've not found a reliable solution. At the moment, I'm letting the Java simulator process write messages to the Output Window. Once we get past the initial release shakedown, I'll consider trapping or logging the messages to reduce the noise.

Can I use QMdbSim2 with a hardware debugger?

Microchip sells a number of hardware debuggers: PICKit, ICE, etc. `QMdbSim2` uses the Microchip SDK to load the software simulator/debugger. A hardware debugger could be loaded using the same SDK interfaces. I might explore hardware debuggers in the future. But, at the moment, no.

Resources

My GitHub QSpice Repository

https://github.com/robdunn4/QSpice

In addition to this project, you'll find information about C-Block component concepts ("C-Block Basics"), various C-Block component examples, tips on using VSCode and Visual Studio IDEs for C-Block development and debugging, and links to other useful QSpice-related sites/repositories.

Microchip

MPLabX IDE (free)

PIC16F15213/14/23/24/43/44 Low Pin Count Micro-Controllers (datasheet)

Microchip Debugger (MDB) User's Guide (one)

Microchip Debugger (MDB)User's Guide (another one)

Simulator Peripheral Support By Device

Microsoft

Visual Studio Downloads (2026 Community Edition is free)

Java JRE/JDK

All users (Basic User, Device Developer, API Developer) must have a 32-bit Java v8 or later runtime (JRE) or JDK (which includes the JRE). I recommend the Eclipse Temurin JDK. This is a bit hard to find on the Adoptium site – you must start by selecting Java v17 or earlier (v17 is the last 32-bit version). Then you must select Windows 32-bit. Then you must click through a "show unsupported versions" option (not a pop-up so it's easy to miss).

Eclipse Adoptium Downloads

Eclipse Adoptium Temurin Releases

Eclipse Adoptium Temrurin v17 32-bit JDK (MSI installer)

Conclusion

I hope that you find this project to be useful or – at least – interesting. Please let me know if you find problems or suggestions for improving the code or this documentation.

You can find me on the QSpice Forums as @RDunn.

--robert